Exp-6 | Error Detection And Correction Using Hamming Code.

26/7/26.

Aim:

To write a program to implement and connection using Hamming code.

Algorithm:

① Start the program

② Read the number of four bit data blocks (n) forming the input data stream

③ for each 4 bit - data block D = $d_1 d_2 d_3 d_4$

Perform Hamming Encoding

④ a) Arrange 7 bit with Parity bit at position 1, 2 and 4 and data bit at position 3, 5, 6 and 7

b) Compute the Parity bit using even Parity:

$P_1 = d_1$ XOR d_2 XOR d_4

$P_2 = d_1$ XOR d_3 XOR d_4

$P_3 = d_2$ XOR d_3 XOR d_4

c) Form the 7 - bit Hamming code word:

$P_1\ P_2\ d_1\ P_4\ d_2\ d_3\ d_4$

⑤ Simulate transmission by introducing a single - bit error at a chosen bit position (entered by the user) in the Code word.

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 3

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 1 1 0
Original Data Bits          : 1110
Encoded Hamming Code (7 bits): 0010110
Enter bit position to introduce error (1-7, 0 = no error): 4
Transmitted Code (with error induced at pos 4): 0011110
Received Code               : 0011110
Checking for errors...
   Error detected at bit position: 4 -> Correcting...
Corrected Code              : 0010110
Recovered Data Bits         : 1110

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 0 1 0
Original Data Bits          : 0010
Encoded Hamming Code (7 bits): 0101010
Enter bit position to introduce error (1-7, 0 = no error): 2
Transmitted Code (with error induced at pos 2): 0001010
Received Code               : 0001010
Checking for errors...
   Error detected at bit position: 2 -> Correcting...
Corrected Code              : 0101010
Recovered Data Bits         : 0010

--- Block 3 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 1 0 1
Original Data Bits          : 1101
Encoded Hamming Code (7 bits): 1010101
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1010111
Received Code               : 1010111
Checking for errors...
   Error detected at bit position: 6 -> Correcting...
Corrected Code              : 1010101
Recovered Data Bits         : 1101
```

5. At receiver, recompute the check bits (syndrome) from the received code word.

$$P_1 = P_1 \text{ XOR } d_1 \text{ XOR } d_2 \text{ XOR } d_4$$

$$c_2 = P_2 \text{ XOR } d_1 \text{ XOR } d_3 \text{ XOR } d_4$$

$$c_4 = P_4 \text{ XOR } d_2 \text{ XOR } d_3 \text{ XOR } d_4$$

6. Compute Error Position = $c_4\ c_2\ c_1$ (binary number formed the check bits)

7. If error Position = 0. no error has occurred
   Else, the bit at "Error Position" is inverted

8. Extract the original 4 data bits from the corrected 7-bit code word

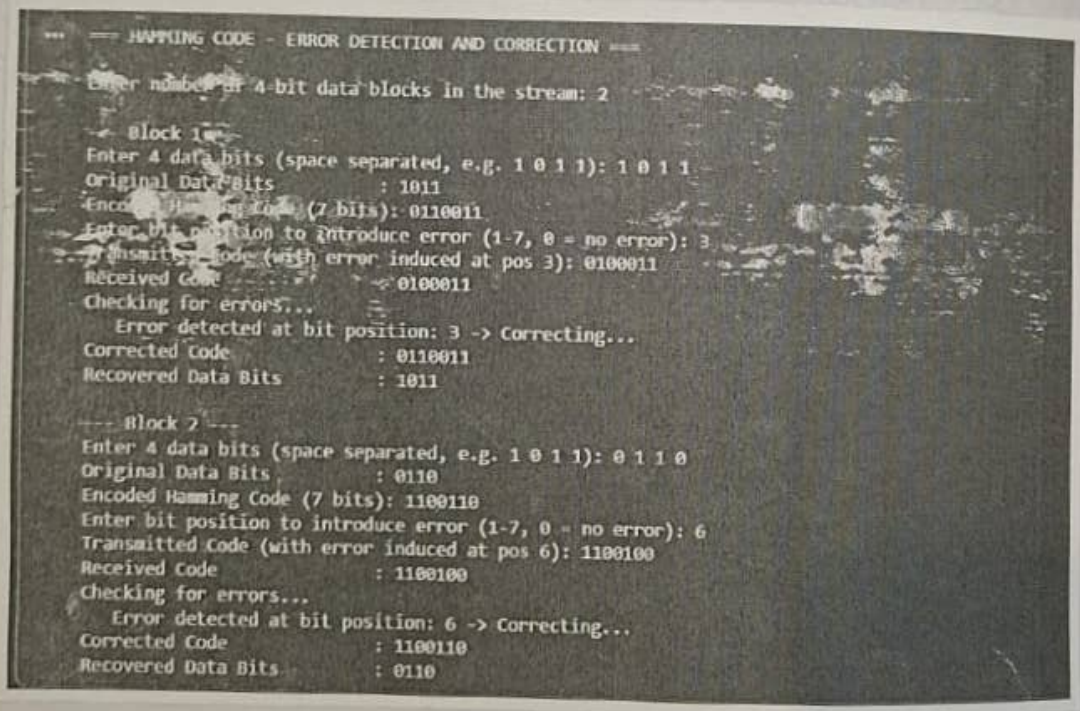
9. Repeat step 3 to 8 for all n blocks in the data stream

10. Stop the program.

PROGRAM

def encode(d):
"d : list of 4 bits [d_1, d_2, d_3, d_4] → returns 7-bit Hamming code list position (1-indexed
"" 1 = P_1 2 = P_2 3 = d_1, 4 = P_4, 5 = d_2, 6 = d_3, 7 = d_4"

```
    u = [0]*7
    u[2] = d[0]   # pos 3
    u[4] = d[1]   # pos 5
    u[5] = d[2]   # pos 6
    u[6] = d[3]   # pos 7
```


=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===
Enter number of 4-bit data blocks in the stream: 2
--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code : 0100011
Checking for errors...
Error detected at bit position: 3 -> Correcting...
Corrected Code : 0110011
Recovered Data Bits : 1011
--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0
Original Data Bits : 0110
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code : 1100100
Checking for errors...
Error detected at bit position: 6 -> Correcting...
Corrected Code : 1100110
Recovered Data Bits : 0110

```
h[0] = h[2] ^ h[4] ^ h[6]      # P1 covers 1,3,5,7
h[1] = h[2] ^ h[5] ^ h[6]      # P2 covers 2,3,6,7
h[3] = h[4] ^ h[5] ^ h[6]      # P4 covers 4,5,6,7

минимум h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4   # хуйня позиция бит
    if error_pos != 0
    print(f"Error detected at bit pos: {error_pos} → correcting ...")
    h[error_pos] ^= 1
else
print("No error detected")
минимум error_pos != 0:
    print(f"Error detected at bit позиции {error_pos} → correcting ...") h[error_pos]

else:
    print("No error detected")
    минимум error_pos

def to_str(bits):
минимум "".join(b) for b in bits)

def main()
    print("== Hamming code - Error detection
    and correction ...")
    h[error_pos - 1] ^= 1
```

OUTPUT:

=== Hamming code - Error detection and correction ===

-- Block 1 --

Enter 4 data bit (span generated eg 1011):

Original data bit : 1011

Encoded Hamming code : 0110011

Enter bit position to introduce error (1-7, 0

Transmitted code (with error induced at Pos 3) : 0110011

Received code : 0100011

Checking for errors...

Error detected at bit position - correct

Corrected code : 0110011

Recovered Data bit : 1011

-- Block 2 --

Enter 4 data bit (span generated, e.g. 1011):

Original data bit : 0110

Encoded Hamming code (7 bit) : 1100110

Enter bit position to introduce error (1-7, = no error) : 6

Transmitted code (with error induced at pos 6) : 1100100

received code : 1100100

Checking for errors --

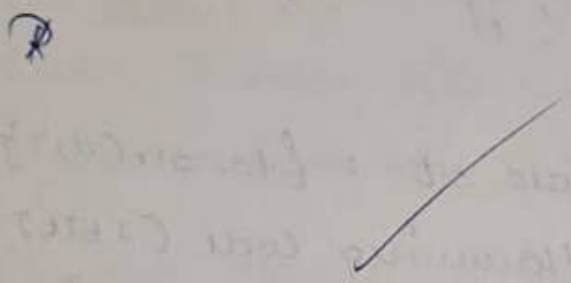
Error detected at bit position : 6

Corrected code : 1100110

```
for b in range(1, n+1):
print(f"\n--- Block {b} ---")
d = list(map(int, input("Enter 4 data
bits (space separated, eg. 1011): ").split()))
h = encode(d)
print(f"Original Data bits : {to_str(d)}")
print(f"Encoded Hamming code (7 bits): {to_
pos = int(input("Enter bit position to
introduce error (1-7, 0 = no error): "))
if 1 <= pos <= 7:
h[pos-1] ^= 1
print(f"Transmitted Code (with error
induced error. (1-7, 0 = no error): "))
if 1 <= pos <= 7:
h[pos-1] ^= 1
print(f'Transmitted Code (with error
induced at pos {pos}): {to_str(h)}")
else:
print(f"Transmitted Code (no error induced
{to_str(h)}")
print(f"Recived code : {to_str(h)}")
print("Checking for error.... ")
d, c = decode(h)
print(f"Corrected Data bits : {to_str(d)}")
if c == 0:
```

Corrected Code : 1100110

Recovered data bit : 0110



Result:

The program to implement error detection and correction using the Hamming code concept was successfully written in Python, executed and executed verified using a sample input data stream.